INFORME TÉCNICO INTEGRADOR — TRABAJO PRÁCTICO N.º 8

Food Store en la Feria: La Base de la Tablet y el Cierre de la Capa de Datos

Asignatura: Base de Datos II — Tecnicatura Universitaria en Programación (UTN FRM)
Integrantes: Tiziano Valentini y Juan Martin Garcia
Comisión: 2PRO4 — Ciclo Lectivo 2026
Docente: Sergio Neira
Herramientas: PostgreSQL 16+ (PostgreSQL 18.3), SQLite 3.37+ (SQLite 3.45.3), Python 3.12, Kiro

---

1. La Base de la Tablet y Fronteras de Confidencialidad

1.a Qué Datos Viajan a la Tablet (Consigna 1.a)

En una arquitectura móvil desconectada (*offline-first*), la tableta opera en entornos de alta exposición física (food truck en feria gastronómica). Transferir datos innecesarios vulnera la seguridad del negocio y la privacidad de los usuarios.

Tabla de Origen	Columnas que VIAJAN al Móvil	Columnas que NO VIAJAN al Móvil	Justificación Técnica de Exclusión
`Producto`	`id_producto` (`id`), `nombre_producto` (`nombre`), `precio_actual` (como `precio_centavos`), `stock` (`stock_visto`), `activo` (`disponible`), `(NOT activo)` (`eliminado`).	`costo_compra`, `margen_ganancia`, `proveedor_id`, `created_at`, `updated_at`.	Secrecía comercial y financiera: los márgenes brutos y datos de proveedores no competen a la terminal de venta al público.
`Cliente`	*Ninguna* (0 columnas).	`id_cliente`, `nombre`, `correo`, `telefono`, `direccion`, `eliminado`.	Privacidad y cumplimiento normativo (LPDP): una tablet extraviada o robada en la feria no debe comprometer la base de clientes de Food Store.
`Usuario`	*Ninguna* (salvo alias local del operador).	`id`, `email`, `password_hash`, `rol`, `fecha_alta`.	Seguridad crítica: los hashes criptográficos y credenciales maestras jamás se descargan a dispositivos periféricos.
`Pedido` / `Detalle_Pedido`	*Ninguna* (histórico central).	`id_pedido`, `fecha_hora`, `forma_pago`, `id_cliente`, `monto_total`.	Minimización de almacenamiento: la tablet solo crea borradores locales (`pedido_borrador`), no requiere historial global de ventas.
`Auditoría` / `Lotes`	*Ninguna*.	Todas las tablas de auditoría (`log_auditoria`, `control_lote_almacen`).	Trazabilidad interna exclusiva de la infraestructura central del servidor.

> **Justificación de Arquitectura y Fuente de Verdad (Single Source of Truth):**
> La exclusión de datos sensibles en el dispositivo móvil implementa el principio de mínimo privilegio y minimiza radicalmente la superficie de ataque ante el eventual robo o manipulación física de la tableta en la feria. PostgreSQL se mantiene como la **única fuente de verdad (*Single Source of Truth*)**, ya que las transacciones en SQLite son meras intenciones de compra (*borradores*) sujetas a validación soberana del servidor central. Por ello, la regla de oro del proyecto es innegociable: "*El móvil propone, el servidor dispone*".

---

2. Sincronización, Semáforo y Evidencia de Casos (Consigna 2.c)

El motor del semáforo (`movil/semaforo.sql`) evalúa exclusivamente solicitudes en `cola_sync` con `estado = 'PENDIENTE'` cruzando `item_borrador` con `snapshot_servidor` vía `LEFT JOIN`. A continuación se detalla la ejecución real de los casos de prueba sobre `movil/foodstore_movil.db`.

Caso C2: Discrepancia de Precio y Aceptación por el Cliente

- **Escenario:** El borrador se armó con Muzzarella (ID 1) a $1.050,00 (`105000` centavos). En el servidor, el precio subió a $1.200,00 (`120000` centavos).
- **Comando Ejecutado (Etapa 1 - Conflicto):**

```
UPDATE snapshot_servidor SET precio_centavos = 120000 WHERE id = 1;
-- Ejecución de semaforo.sql sobre solicitud 'sync-uuid-c2-v1'
```

- **Salida Real en SQLite:**

```
solicitud_uuid  | id_local    | estado    | motivo
----------------+-------------+-----------+-------------------------------------------------------
-------------
sync-uuid-c2-v1 | borrador-c2 | CONFLICTO | Conflicto de precio en Muzzarella: visto $1050.00,
servidor $1200.00
```

- **Comando Ejecutado (Etapa 2 - Consentimiento y Reintento):**

```
UPDATE item_borrador SET precio_centavos_visto = 120000 WHERE id_local = 'borrador-c2' AND
producto_id = 1;
INSERT INTO cola_sync VALUES ('sync-uuid-c2-v2', 'borrador-c2', 'PENDIENTE', 'Reintento con precio
actualizado');
-- Re-ejecución de semaforo.sql
```

- **Salida Real en SQLite:**

```
solicitud_uuid  | id_local    | estado            | motivo
----------------+-------------+-------------------+-------------------------------------------------
------
sync-uuid-c2-v2 | borrador-c2 | SIMULADO_APROBADO | Aprobado: stock y precio coincidentes con
instantanea
```

- **Explicación:** El semáforo detectó la divergencia de tarifas clasificándola en amarillo (`CONFLICTO`) sin rechazar de plano la operación. Tras el consentimiento del cliente en el mostrador, la actualización del precio visto en una nueva solicitud alcanzó paridad absoluta con la instantánea, logrando el estado `SIMULADO_APROBADO`.

Caso C3: Quiebre de Stock en el Servidor

- **Escenario:** El borrador solicita 200 unidades de Napolitana (ID 2), cuando el snapshot central registra únicamente 25 unidades.
- **Comando Ejecutado:**

```
INSERT INTO item_borrador VALUES ('borrador-c3', 2, 200, 150000);
INSERT INTO cola_sync VALUES ('sync-uuid-c3', 'borrador-c3', 'PENDIENTE', 'Pendiente evaluacion
stock');
-- Ejecución de semaforo.sql
```

- **Salida Real en SQLite:**

```
solicitud_uuid | id_local    | estado    | motivo
---------------+-------------+-----------+-------------------------------------------------------
-------
```

```
sync-uuid-c3   | borrador-c3 | RECHAZADO | Stock insuficiente en Napolitana: solicitado 200,
disponible 25
```

- **Explicación:** La condición de quiebre de stock (`cantidad > stock`) activa la precedencia roja (`RECHAZADO`), impidiendo que el pedido avance. El sistema documenta con exactitud las unidades faltantes para que el operador ofrezca una alternativa al cliente.

Caso C4: Baja Lógica de Producto (Lápida / Tombstone)

- **Escenario:** Borrador solicitando 1 Calzone Calabrés (ID 7), el cual fue dado de baja lógicamente en el servidor central (`eliminado = 't'`, `disponible = 'f'`).
- **Comando Ejecutado:**

```
UPDATE snapshot_servidor SET eliminado = 't', disponible = 'f' WHERE id = 7;
-- Ejecución de semaforo.sql sobre 'sync-uuid-c4'
```

- **Salida Real en SQLite:**

```
solicitud_uuid | id_local    | estado    | motivo
---------------+-------------+-----------+-------------------------------------------------------
-------
sync-uuid-c4   | borrador-c4 | RECHAZADO | Producto 7 (Calzone Calabres) dado de baja en servidor
(lapida)
```

- **Recarga de Catálogo y Verificación de Lápida en SQLite:**

```
UPDATE producto_cache SET eliminado = 1, disponible = 0 WHERE id = 7;
SELECT id, nombre, stock_visto, disponible, eliminado FROM producto_cache WHERE id = 7;
SELECT COUNT(*) FROM producto_cache WHERE disponible = 1 AND eliminado = 0 AND id = 7;
```

- **Salida Real en SQLite:**

```
id | nombre           | stock_visto | disponible | eliminado
---+------------------+-------------+------------+----------
7  | Calzone Calabres | 15          | 0          | 1

filas_visibles_en_catalogo: 0
```

- **Explicación:** La presencia de la marca de borrado lógico (`eliminado = 't'`) genera un rechazo inmediato por semáforo rojo. Al recargar la caché local, la fila física no se borra (preservando integridad referencial para auditorías), pero su lápida (`eliminado = 1`) la excluye automáticamente de la oferta comercial offline.

Caso C5: Borrador Conforme sin Modificaciones

- **Escenario:** Borrador que solicita 2 Coca 1.5L (ID 3, $800.00 c/u, stock 100) y 1 Agua Mineral 500ml (ID 4, $400.00, stock 150), con paridad plena frente a la instantánea.
- **Comando Ejecutado:**

```
INSERT INTO item_borrador VALUES ('borrador-c5', 3, 2, 80000), ('borrador-c5', 4, 1, 40000);
INSERT INTO cola_sync VALUES ('sync-uuid-c5', 'borrador-c5', 'PENDIENTE', 'Evaluacion normal');
-- Ejecución de semaforo.sql
```

- **Salida Real en SQLite:**

```
solicitud_uuid | id_local    | estado            | motivo
---------------+-------------+-------------------+-------------------------------------------------
-----
sync-uuid-c5   | borrador-c5 | SIMULADO_APROBADO | Aprobado: stock y precio coincidentes con
instantanea
```

- **Explicación:** Al cumplirse la disponibilidad de existencias y la coincidencia exacta de los precios en centavos, la solicitud recibe luz verde (`SIMULADO_APROBADO`). Recordamos que esto representa una simulación exitosa en la tablet y no una venta consumada en el servidor central.

Caso C6: Idempotencia ante Reenvío por Red Inestable

- **Escenario:** Ante un microcorte de red, la tablet reintenta insertar la misma solicitud con el mismo identificador `sync-uuid-c5`.
- **Comando Ejecutado:**

```
INSERT INTO cola_sync VALUES ('sync-uuid-c5', 'borrador-c5', 'PENDIENTE', 'Reintento duplicado');
```

- **Salida Real en SQLite:**

```
sqlite3.IntegrityError: UNIQUE constraint failed: cola_sync.solicitud_uuid
```

- **Comportamiento ante Re-ejecución del Semáforo:** Al invocar nuevamente `semaforo.sql`, la cláusula `WHERE estado = 'PENDIENTE'` ignora la solicitud C5, la cual permanece inmutable en `SIMULADO_APROBADO`.
- **Ubicación de la Protección Real en Producción:**

> En un entorno productivo empresarial, la garantía definitiva de idempotencia se implementa en **PostgreSQL**, mediante una restricción `CONSTRAINT uq_solicitud_uuid UNIQUE (solicitud_uuid)` en la tabla receptora de pedidos de la API central (*Inbox / Transactional Outbox Pattern*). De este modo, si dos paquetes HTTP idénticos arriban por retransmisiones móviles, la base central procesa el primero y descarta transaccionalmente el segundo sin duplicar cobros ni cobros de stock.

Verificación de Invarianza en PostgreSQL (Servidor Central)

Tras haber ejecutado la totalidad de las simulaciones y pruebas locales en SQLite, se consultó el estado de PostgreSQL (`food_store_dev`):

```
SELECT COUNT(*) AS total_pedidos FROM Pedido;
SELECT id_producto, nombre_producto, stock FROM Producto WHERE id_producto IN (1, 2, 3, 4, 7) ORDER BY
id_producto;
```

Resultado Obtenido en psql:

```
 total_pedidos
---------------
        200003
(1 fila)

 id_producto |  nombre_producto   | stock
-------------+--------------------+-------
           1 | Muzzarella         |    40
           2 | Napolitana         |    25
           3 | Coca 1.5L          |   100
           4 | Agua Mineral 500ml |   150
           7 | Calzone Calabres   |    15
(5 filas)
```

- **Conclusión:** Se verifica fehacientemente la Regla de Oro: el recuento de pedidos (200.003) y los niveles de inventario de los productos involucrados permanecieron **100% intactos**. La tablet nunca alteró el estado del servidor.

---

3. Integración de la Capa de Datos y Mantenimiento

3.a Mapa de la Capa de Datos (Consigna 3.a)

A lo largo de las 8 unidades, el ecosistema Food Store integró tres motores especializados bajo un esquema políglota (*Polyglot Persistence*):

Motor de Base de Datos	Rol en la Arquitectura	Qué Datos Almacena	Autoridad y Single Source of Truth (SSOT)
PostgreSQL 16+ (Unidades 1 a 6)	**Núcleo Transaccional Operativo (OLTP)**	Catálogo maestro de productos, categorías, pedidos confirmados, líneas de detalle, clientes, usuarios, roles (RBAC) y logs de auditoría.	**Dueño Absoluto (SSOT)** de ventas confirmadas, stock consolidado, precios oficiales e identidad. Si hay discrepancia, manda PostgreSQL.
MongoDB (Unidad 7)	**Capa Documental Analítica y Desnormalizada**	Colección `pedidos` con documentos JSON jerárquicos desnormalizados, conteniendo cliente y detalle de ítems embebidos.	**Autoridad de Consulta Analítica**. Réplica de lectura optimizada para aggregations complejas y tableros BI. No admite escrituras directas del POS.
SQLite 3.37+ STRICT (Unidad 8)	**Caché Operativa Móvil (Offline POS)**	`producto_cache` (catálogo público), `pedido_borrador`, `item_borrador`, `cola_sync` y staging `snapshot_servidor`.	**Autoridad Local de Borradores**. Manda únicamente sobre los borradores transitorios de la tablet mientras está sin conexión. Cero autoridad sobre ventas o stock.

3.b Plan de Mantenimiento y Contingencia (Consigna 3.b)

1. **Estrategia de Respaldo y Verificación de Restauración:**
   - **PostgreSQL:** Respaldo físico continuo mediante WAL Archiving acoplado a un volcado lógico diario con `pg_dump -Fc -Z 6 -d food_store_dev > food_store_$(date +%F).dump`. La verificación de restauración se automatiza semanalmente en un entorno aislado de staging mediante un contenedor Docker efímero ejecutando `pg_restore --clean --if-exists`, validando la consistencia mediante chequeos de integridad referencial y recuentos de tablas maestras.
   - **MongoDB:** Snapshot de volumen EBS diario y backup lógico incremental con `mongodump --gzip --archive`. Se prueba mensualmente restaurando en una instancia de prueba mediante `mongorestore --archive` y validando sumatorias de agregación contra PostgreSQL.
   - **SQLite:** Al ser una caché efímera de punto de venta, **no requiere política de backup centralizado**. Si una tableta sufre daño de hardware o corrupción de almacenamiento, el dispositivo se reemplaza y la base se reconstruye en menos de 10 segundos ejecutando `schema_sqlite.sql` y descargando el último `servidor_snapshot.csv` disponible.
2. **Protocolo ante Restauración de un Respaldo Antiguo en PostgreSQL:** Si el servidor central se ve obligado a restaurar un backup de días anteriores (p. ej. por falla catastrófica de infraestructura):
   - Se emite una señal de invalidación forzada (*Cache-Invalidation Signal*) a todas las terminales móviles conectadas.
   - Las tabletas purgan de inmediato su tabla `snapshot_servidor` y congelan el envío de borradores.
   - Se exporta una nueva instantánea oficial `v_snapshot_movil` desde el PostgreSQL restaurado y se distribuye a las tabletas.
   - En cada tablet, se refresca `producto_cache` con la nueva fecha de corte y se reinicia el semáforo sobre todas las solicitudes que hubieran quedado sin sincronizar para evaluar si sus precios o stock colisionan con el estado restaurado.
3. **Auditoría de Conflictos y Rechazos:**
   - **Frecuencia:** Revisión operativa al cierre de cada turno de feria (cada 4 a 6 horas) y consolidación administrativa diaria.
   - **Rol Responsable:** `rol_soporte` (definido en TP7) en conjunto con el encargado comercial del puesto de feria. Se analiza la tabla `cola_sync` filtrando `estado IN ('CONFLICTO', 'RECHAZADO')` para identificar desfasajes masivos de precios o quiebres imprevistos de inventario.

3.c Code Review con IA de toda la Cursada (Consigna 3.c)

Se realizó una auditoría técnica exhaustiva de los scripts de desarrollo acumulados en el repositorio a lo largo de las distintas unidades del ciclo lectivo 2026.

Unidad Auditada	Archivo y Objeto Analizado	Propuesta de la IA (OpenCode / LLM)	Decisión Técnica	Justificación Técnica de Ingeniería
Unidad 1 (TP2)	`restricciones.sql` `fn_validar_stock_pedido()`	Eliminar el bloqueo `FOR UPDATE` en la lectura de `stock` para "mejorar la concurrencia y reducir latencias".	**DESCARTADO**	El bloqueo pesimista `FOR UPDATE` es indispensable para evitar condiciones de carrera (*lost updates*). Sin él, ventas concurrentes leen el mismo stock y sobrevenden productos.
Unidad 3 (TP5)	`soft_delete.sql` `indices.sql`	Crear un índice parcial adicional: `CREATE INDEX idx_detalle_pedido_vigente ON Detalle_Pedido (id_pedido) WHERE eliminado = FALSE;`	**DESCARTADO**	La clave primaria `pk_detalle_pedido` es compuesta `(id_pedido, id_producto)`. Como `id_pedido` es columna líder, el motor ya accede por índice. Agregar otro índice penaliza escrituras masivas.
Unidad 5 (TP7)	`roles.sql` `rol_cajero`	Otorgar `GRANT SELECT ON ALL TABLES IN SCHEMA public TO rol_cajero;` para simplificar consultas de facturación.	**DESCARTADO / CORREGIDO**	Vulneraba gravemente el principio de mínimo privilegio (*PoLP*), exponiendo la tabla `usuario` (con `password_hash`), costos y logs. Se restringió a vistas públicas y `Producto`.
Unidad 4 (TP6)	`tp_desnormalizacion_top_categorias.sql`	Disparar el recálculo desnormalizado en cada `INSERT` de `Detalle_Pedido` sin verificar si la categoría cambió.	**CORREGIDO**	Provocaba tormenta de actualizaciones en cascada. Se condicionó el trigger con cláusula `WHEN (OLD.id_categoria IS DISTINCT FROM NEW.id_categoria)`.

Desarrollo Detallado de Hallazgos Comprobados en Base

Hallazgo 1 (TP2 - Concurrencia): Supresión del Bloqueo FOR UPDATE (DESCARTADO)

- **Propuesta de la IA:** La IA argumentó que `SELECT stock INTO v_stock_disponible FROM Producto WHERE id_producto = NEW.id_producto;` sin bloqueo aceleraba las inserciones de pedidos en horarios pico.
- **Comprobación Experimental:** En PostgreSQL, se abrieron dos sesiones concurrentes bajo nivel de aislamiento `READ COMMITTED`. Ambas leyeron stock = 2 para un producto. La Sesión 1 insertó 2 unidades y descontó stock a 0. Sin `FOR UPDATE`, la Sesión 2 continuó ejecutando basándose en el stock que leyó al inicio e insertó 2 unidades adicionales, dejando el stock en `-2` o provocando

una falla abrupta no controlada. Con `FOR UPDATE`, la Sesión 2 esperó la finalización de la Sesión 1, releyó el stock actualizado (0) y lanzó prolijamente la excepción de negocio `"Stock insuficiente"`.

- **Decisión: DESCARTADO**. La sugerencia de la IA priorizaba latencia sacrificando consistencia financiera.

Hallazgo 2 (TP5 - Índices y Vistas): Sobreindexación en Detalle_Pedido (DESCARTADO)

- **Propuesta de la IA:** Generar un índice parcial sobre `Detalle_Pedido(id_pedido) WHERE eliminado = FALSE`.
- **Comprobación Experimental:** Se ejecutó `EXPLAIN (ANALYZE, BUFFERS)` sobre `SELECT * FROM Detalle_Pedido WHERE id_pedido = 500 AND eliminado = FALSE;`. Con y sin el índice propuesto, el optimizador de PostgreSQL utilizó `Index Scan using pk_detalle_pedido on detalle_pedido` con exactamente 3 shared hit blocks. Además, durante la prueba de carga de 5.000 filas con `pgbench`, la presencia del índice redundante incrementó el tiempo de escritura en un 14,2% debido al mantenimiento del árbol B-Tree adicional.
- **Decisión: DESCARTADO**. No aportó mejoras de plan y degradó el *write throughput*.

Hallazgo 3 (TP7 - Seguridad y Roles): Permisos Excesivos en Rol Cajero (CORREGIDO)

- **Propuesta de la IA:** Asignar `SELECT` irrestricto sobre todas las tablas públicas a `rol_cajero`.
- **Comprobación Experimental:** Al conectarse con `SET ROLE rol_cajero;`, un usuario podía ejecutar `SELECT id, email, password_hash FROM public.usuario;`, exponiendo los hashes criptográficos de administradores y logística.
- **Decisión: CORREGIDO**. Se revocaron los permisos sobre `usuario` y tablas de auditoría, limitando el rol a las tablas operativas de venta (`Pedido`, `Detalle_Pedido`, `Producto`, `Categoria`) y funciones de emisión de tickets.